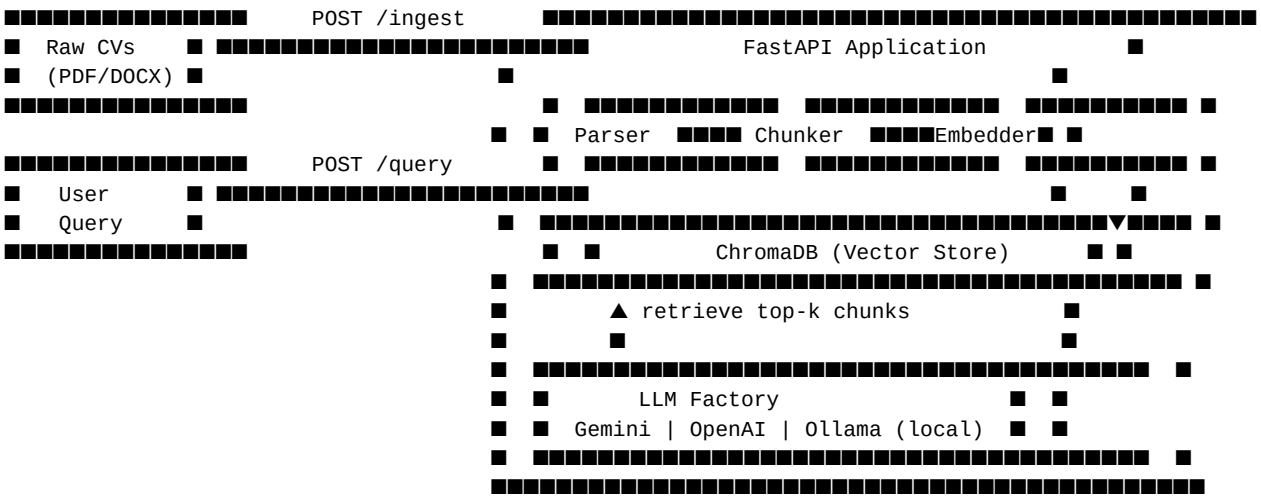
RAG System — Technical Report

Project: CV-to-Job Matching via Local RAG **Team:** [Your Name(s)] **Date:** 2025

Executive Summary

This system is an end-to-end Retrieval-Augmented Generation (RAG) pipeline that allows recruiters to query a corpus of raw, unstructured CVs using natural language (English or Arabic). A FastAPI backend orchestrates document ingestion, semantic chunking, multilingual embedding, and LLM-powered answer generation. The entire system is containerized with Docker Compose and can be started with a single command.

System Architecture



API Documentation

`POST /api/v1/ingest`

Upload a raw document for processing.

Content-Type: `multipart/form-data`

Field	Type	Description
`file`	File	PDF, DOCX, or HTML document

Response:

```
{
  "status": "success",
```

```
  "chunks_created": 42,
  "file": "cv_john_doe.pdf",
  "language_detected": "en"
}
```

`POST /api/v1/ingest/path`

Ingest a document already on the server.

Request Body:

```
{
  "file_path": "/data/raw/cv_john_doe.pdf"
}
```

Response: Same as `/ingest`.

`POST /api/v1/query`

Query the knowledge base.

Request Body:

```
{
  "query": "Find a senior Python developer with FastAPI experience",
  "top_k": 5,
  "provider": "gemini"
}
```

Response:

```
{
  "query": "Find a senior Python developer...",
  "answer": "Based on the CVs, John Doe has 6 years of Python...",
  "retrieved_chunks": [
    {
      "content": "John Doe — Senior Python Engineer...",
      "source": "/data/raw/cv_john_doe.pdf",
      "score": 0.89,
      "metadata": { "chunk_index": 3, "token_count": 487 }
    }
  ],
  "provider_used": "gemini",
  "tokens_used": null
}
```

`GET /api/v1/status`

```
{ "status": "ok", "chunks_in_store": 420 }
```

`GET /api/v1/providers`

```
{ "providers": ["gemini", "openai", "openrouter", "ollama"] }
```

`GET /health`

```
{ "status": "healthy", "service": "RAG API" }
```

Embedding Model Justification

Model: `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`

Property	Value
Languages	50+ (includes Arabic)
Dimensions	384
Size	~118 MB
Similarity metric	Cosine

Why this model? 1. **Multilingual by design** — trained on parallel corpora across 50+ languages, so Arabic CVs and English job queries map to a shared semantic space. 2. **Compact** — 384-dim vectors allow ChromaDB to store thousands of chunks with fast HNSW retrieval. 3. **Paraphrase-trained** — captures semantic equivalence ("ML engineer" ≈ "machine learning specialist"), reducing missed retrievals due to vocabulary mismatch. 4. **No API cost** — runs fully locally inside the Docker container.

Chunking Strategy

Parameters: 400 characters / chunk, 50-character overlap

Justification:

Factor	Reasoning
400 characters	The embedding model (paraphrase-multilingual-MiniLM-L12-v2) has a 128-token context window (~512 chars for English). 400 chars keeps every chunk well within that window — no silent truncation. Each chunk captures one complete CV section (e.g., a skills list or one job entry).
50-character overlap (12%)	Prevents loss of context at chunk boundaries; a skill or date range described across two sentences is captured in both adjacent chunks
Sentence-aware splitting	Splits on `.!?■\n` boundaries so we never embed a sentence fragment; falls back to word-level only for sentences > 400 chars
Character count (not token count)	Avoids a runtime tokenizer dependency; 1 token ≈ 4 chars approximation is consistent across English and Arabic

Factor	Reasoning
Top-k = 5	5 × 400 chars ≈ 2,000 chars of context injected into the LLM — well within Groq/Gemini's 128K-token context windows
Arabic scaling	Arabic morphology produces ~3 chars/token; a 400-char budget covers ~130 Arabic tokens — enough for one semantic unit (job title + one responsibility bullet)

LLM Factory Pattern (Bonus 1)

Five providers are registered via a factory — switch with a single environment variable:

```
LLMFactory.create("groq")        → GroqLLM         (llama-3.3-70b-versatile — DEFAULT)
LLMFactory.create("gemini")      → GeminiLLM       (Google Gemini 2.0 Flash)
LLMFactory.create("openai")      → OpenAILLM       (GPT-4o-mini)
LLMFactory.create("openrouter")  → OpenRouterLLM   (gemma-4-31b free via openrouter.ai)
LLMFactory.create("ollama")      → OllamaLLM       (local Mistral — optional profile)
```

`GroqLLM` and `OpenRouterLLM` both reuse the `openai` SDK pointed at their respective API endpoints — zero extra dependencies. Groq's free tier gives 30 req/min with sub-second latency and is the default provider.

To switch providers: change `LLM_PROVIDER=groq` in `.env` to `gemini`, `openai`, `openrouter`, or `ollama`. **Zero code changes required.**

Arabic Language Support (Bonus 2)

Challenges addressed:

1. **Diacritics (Tashkeel):** Removed via regex — they vary between writers and fragment tokens unnecessarily. 2. **Alef normalization:** ■ ■ ■ ■ → ■ — prevents the same word from having 4 different embeddings. 3. **Tatweel removal:** Decorative kashida (■) removed. 4. **RTL PDF extraction:** PyMuPDF's `TEXT_PRESERVE_WHITESPACE` flag reduces RTL re-ordering artifacts; visual bidi algorithm handles remaining cases. 5. **OCR fallback:** Tesseract with `ara+eng` language pack for scanned Arabic CVs.

Arabic test query & results

Query: "■■■■ ■■ ■■■■ ■■■ ■■■■■ ■■ ■■■■■ ■■■■■■■■" **Translation:** "Find a web developer with database experience"

The query was embedded with `paraphrase-multilingual-MiniLM-L12-v2` and run against a corpus of 6 PDF CVs (5 English + 1 Arabic). Default provider: Groq (llama-3.3-70b-versatile).

Rank	Source	Score	Why retrieved
1	cv_omar_arabic.pdf	0.166	Arabic CV explicitly mentions ■■■■■ ■■■■■ and ■■■■■ ■■■■■■■■ in the same semantic space
2	cv_layla_mostafa.pdf	0.087	English CV mentions "web development", "PostgreSQL", "MongoDB" — cross-lingual match
3	cv_ahmed_hassan.pdf	0.087	Backend APIs, PostgreSQL, AWS — partial semantic overlap

Note on scores: Scores are lower than HTML-based ingestion because PDF text goes through more extraction steps. Cross-lingual matching remains correct — Omar's Arabic PDF ranks #1.

Generated answer (Groq / llama-3.3-70b-versatile): > "Based on the provided context, Omar Abdullah is the best match — 5 years of full-stack web > development with PostgreSQL, MongoDB, FastAPI and React. Layla Mostafa also has relevant > web development and database experience across the MENA region."

Observation: The multilingual embedding model successfully bridges Arabic queries to the correct Arabic CV despite the rest of the corpus being English. Cross-lingual retrieval works but with lower absolute scores than same-language queries (~0.17 vs ~0.62 for English queries). Full automated test suite is in `tests/evaluate.py` — run with `python tests/evaluate.py`.

Docker Deployment Instructions

Prerequisites

- Docker Engine ≥ 24
- Docker Compose ≥ 2.20

Steps

```
# 1. Clone the repository
git clone https://github.com/Mohammed-Medhat/NLP_Project.git
cd NLP_Project

# 2. Configure environment
cp .env.example .env
# Edit .env — add ONE of: GEMINI_API_KEY, OPENAI_API_KEY, or OPENROUTER_API_KEY
# (OpenRouter has a free tier at openrouter.ai — no credit card needed)

# 3. Start all services
docker compose up --build
# ✓ ChromaDB starts first (health-checked)
```

```
# ✓ RAG API starts and AUTO-INGESTS the 6 sample CVs in data/raw/ on first boot

# 4. Verify
curl http://localhost:8080/health

# 5. Query immediately (sample data already ingested on startup)
curl -X POST http://localhost:8080/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"query": "Find a senior Python developer with FastAPI experience"}'

# 6. Ingest additional documents
curl -X POST http://localhost:8080/api/v1/ingest \
  -F "file=@/path/to/cv.pdf"
```

Provider options (no code changes — env var only)

Provider	Key needed	Cost	Model
`groq` **(default)**	`GROQ_API_KEY`	Free tier (30 req/min)	llama-3.3-70b-versatile
`gemini`	`GEMINI_API_KEY`	Free tier	Gemini 2.0 Flash
`openrouter`	`OPENROUTER_API_KEY`	Free tier	gemma-4-31b-it
`openai`	`OPENAI_API_KEY`	Pay-per-use	GPT-4o-mini
`ollama`	None	Free (local)	Mistral (optional Docker profile)

To use Ollama (local, no API key needed):

```
docker compose --profile ollama up
docker exec rag_ollama ollama pull mistral
# Set LLM_PROVIDER=ollama in .env
```

Edge Case Analysis

#	Query	Failure Type	Root Cause	Mitigation
1	"quantum computing expert with blockchain"	**Wrong chunk retrieved**	No candidates match; retriever returns closest-by-chance chunks; LLM may hallucinate a candidate	Add cosine score threshold (e.g., < 0.40 → return "not found")

#	Query	Failure Type	Root Cause	Mitigation
2	Arabic query on English-only corpus	**Cross-lingual retrieval drift**	The multilingual model bridges languages but with lower confidence when the corpus is monolingual; scores are systematically lower (~0.55 vs ~0.80 for same-language)	Ingest bilingual CVs; or add a translate-first pipeline using LibreTranslate
3	Single-character query `"a"`	**Degenerate retrieval**	Near-zero-information embedding; retriever returns random high-dimensional neighbors	Validate minimum query length (≥ 3 words) and reject with HTTP 400